

Module - Plateformes de Développement

Spring

Support de cours

Par :

Dr. Ing. Faouzi JAIDI

faouzi.jaidi@gmail.com

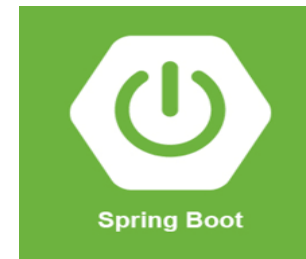
2021-2022

Plan

- 1. Introduction
- 2. Maven
- 3. Logs : Log4J
- 4. Tests: JUnit
- 5. Tests & Qualité : Sonar
- 6. IOC & ID
- 7. Spring Boot
- 8. Spring Data JPA
- 9. Spring MVC REST
- 10. JSF
- 11. AOP
- 12. Projet
- 13. Spring Security
- 14. Extensions

Spring

Spring AOP



Introduction:

□ AOP

- Programmation Orientée Aspect «POA» [*Aspect Oriented Programming «AOP»*]
- Permet d'enrichir des classes ou des méthodes existantes en **rajoutant des comportements** :
 - traces (logs),
 - gestion des transactions,
 - gestion de la sécurité,
 - monitoring,
 - ...

➤ *Il s'agit de problématiques transverses techniques.*

Introduction:

□ Spring AOP

- La Programmation Orientée Aspect n'est pas une notion propre au Spring Framework.
- Dans la communauté Java, le projet *AspectJ* est le projet le plus avancé pour intégrer la programmation orientée aspect au langage.
- *Spring AOP* est le module Spring chargé d'ajouter le support de la programmation aspect en se basant en grande partie sur AspectJ.

AOP

□ AOP vs IOC/ID

- Deux concepts principaux de Spring :
 - Inversion de Contrôle IOC – Injection de Dépendance ID
 - AOP
- IOC / ID : **injecter** des beans dans des objets Java.
- AOP : **enrichir** les objets Java par des aspects.
 - *Un **aspect** représente une catégorie d'actions à réaliser dans certaines conditions.*
 - *La POA est utilisée pour implémenter des fonctionnalités transverses (cross-cutting concerns).*

AOP

□ Programmation : Procédurale / Orientée Objets / Orientée Aspects

<i>Programmation</i>	<i>Concepts de base</i>	<i>Préoccupation</i>
Procédurale	Procédures et Fonctions	Découpage du code en portions
Orientée Objets « OOP »	Objets / Classes	Données sous forme d'objets
Orientée Aspects « AOP »	Aspects	Fonctionnalités transverses « <i>Cross-Cutting Concerns</i> »

□ Problématique : *Cross Cutting Concerns*

- Exemple de “cross cutting concerns” : *tracing, caching, transaction, security, performance monitoring, Error handling*.
- Impact des “cross-cutting concerns” sur notre système (sans AOP):
 - **Tangling** : Mélange du code métier avec du code de (sécurité, traçage ...).
 - **Scattering** : Duplication d’un bout de code dans plusieurs endroits. (tel que test de sécurité).
 - *Problème de maintenabilité.*
 - *L’AOP peut être une solution à ces problèmes (Tangling, Scattering).*

AOP

□ SoC / DRY / Crosscutting Concerns

- *Separation of Concerns (SoC)* : Au lieu d'avoir un appel direct à un module technique depuis un module métier, en AOP, le code du module en cours de développement est concentré sur le but poursuivi (la logique métier).
- L'AOP permet d'enrichir ce code métier avec ces fonctionnalités transversale (*Crosscutting Concerns*).
 - Exemple : Ajout de logs dans une application existante.
- *Don't Repeat Yourself (DRY)* : éviter la duplication de code.

□ Avantages et Inconvénients

- (+) Facilité de maintenance, puisque les fonctionnalités transverses sont regroupées dans les aspects.
- (+) Particulièrement adapté pour les fonctionnalités techniques.
- (+) Permet une meilleure modularité du code et des applications ce qui augmente la réutilisation du code et la modularité des systèmes.
- (-) La lecture du code contenant les traitements ne permet pas de connaître les aspects qui seront exécutés (sans utiliser un outil).
- (-) Nécessite un temps de prise en main.

AOP

□ Implémentation

- L'AOP peut être utilisée :
 - Indirectement: utilisation des annotations Spring, tel que, @Configuration et @Transactional.
 - Directement : pour mettre en œuvre ses propres Aspects; Spring facilite cette mise en œuvre
- L'AOP peut être mise en œuvre via **Spring AOP** ou **AspectJ** (qui a sa propre syntaxe).
- L'AOP utilise le *Design Pattern* **Proxy**.

AOP

- Un proxy est une classe se substituant à une autre classe.
- Le proxy implémente la même interface que la classe à laquelle il se substitue.
- Par exemple: Spring va créer une classe “proxy” qui implémente IEntrepriseService et va l’injecter à la place du bean “EntrepriseService”.
- Cette classe proxy contient les aspects et les méthodes de l’interface.

- La POA introduit un vocabulaire spécifique
 - **Point de jonction (JoinPoint)**: l'endroit où l'on veut qu'un aspect s'applique; comme l'appel d'une méthode ou le lancement d'une exception
 - **Coupe (Pointcut)**: une expression, qui permet de sélectionner plusieurs Join points. Exemple: toutes les méthodes public dans un package précis.
 - **Greffon (Advice)**: l'action particulière de l'aspect à exécuter quand le programme atteint le point de jonction (le code que l'on veut rajouter). On peut ajouter ce code avant, après, autour de la méthode, etc.
 - **Aspect**: la problématique spécifique que l'on veut ajouter transversalement. Une classe (annotée avec **@Aspect**) qui encapsule une fonctionnalité transverse et est composée d'un ou de plusieurs **Pointcut** et **Advice**.
 - **Weaving (tissage)** : action d'insertion des aspects (fait par Spring AOP).

□ Types d'Advice

- Il est possible de définir 5 types d'advices
 - **Before advice** : s'exécute avant le Join point. S'il lance une Exception, le Join point ne sera pas appelé
 - **After returning advice** : s'exécute après le Join point, si celui-ci s'est bien exécuté (s'il n'y a pas d'Exception)
 - **After throwing advice** : s'exécute si une Exception a été lancée pendant l'exécution du Join point
 - **After advice** : s'exécute après le Join point, qu'il y ait eu une Exception ou non
 - **Around advice** : s'exécute autour du Join point. C'est l'advice le plus puissant.

AOP

□ Point Cut

- Une **Coupe (Pointcut)** commence par un *designator* qui précise la cible définit.
- *Designators* supportés par Spring AOP :

<i>Designator</i>	<i>Cible</i>
execution()	L'appel à une méthode décrite par la coupe
within()	L'appel à une méthode d'un type décrit par la coupe
target()	L'appel à une méthode d'un type donné par la coupe
args()	L'appel à une méthode dont les types des paramètres correspondent à ceux donnés par la coupe
@target()	L'appel à une méthode d'un type portant l'annotation donnée par la coupe
@args()	L'appel à une méthode dont les arguments portent les annotations données par la coupe
@within()	L'appel à une méthode d'un type portant l'annotation décrite par la coupe
@annotation()	L'appel à une méthode portant elle-même l'annotation décrite par la coupe

□ Advice

"execution(Modifiers-pattern? Ret-type-pattern Declaring-type-pattern?Name-pattern(param-pattern) Throws-pattern?)"

- ❖ "?" veut dire optionnel
- ❖ **Modifiers-pattern?** : public, private ...
- ❖ **Ret-type-pattern** : le type de retour.
- ❖ **Declaring-type-pattern?** : nom de la classe y compris le package.
- ❖ **Name-pattern** : nom de la méthode.
- ❖ **Throws-pattern?** : l'exception.
- ❖ **".."** : 0 ou plusieurs paramètres

Exercice

- Analyser le code suivant et préciser l'Aspect, le JoinPoint, le PointCut, l'Advice et le type d'advice :

```
import .....
@Component
@Aspect
public class LoggingAspect {

    private static final Logger logger =
    LogManager.getLogger(LoggingAspect.class);

    @Before("execution(* tn.enicarthage.service.*.*(..))")
    public void logMethodEntry(JoinPoint joinPoint) {
        String name = joinPoint.getSignature().getName();
        logger.info("In method " + name + " : ");
    }
}
```

Exercice

➤ *Expliquer les PointCut suivants :*

- @Before("execution(* tn.enicarthage.service.*.*(..))")
- @Before("execution(public * *(..))")
- @Before("execution(* set*(..))")
- @Before("execution(* tn.enicarthage.*.*(..))")



Spring

TP

